TERMINAL 3 · AGENT DEVELOPER KIT · INTEGRATION REPORT

Quickstart and Walkthrough, completed — and sixteen bugs found on the way

A Rust TEE contract compiled to a WASM component, registered on T3N testnet, authorized to an agent identity, and invoked end to end — with the enclave reaching a live external API. Along the way, the published Quickstart turned out not to run.

SUBMITTED BY	Hitansh Gopani — AI/ML Engineer
CONTACT	gopanihitansh5@gmail.com · x.com/Hitansh54
DATE	8 August 2026
AGENT DID	did:t3n:6ec29eeb5cb122d05e006391d2c954b2390032ed
CONTRACT	z:6ec29eeb…32ed:flight @ 0.4.1 · contract_id 511
REPOSITORY	github.com/gopanihitansh5-collab/t3n-adk-submission
STACK	@terminal3/t3n-sdk 4.30.0 · Node 24.14.1 · Rust 1.97.1 · wasm32-wasip2

THE SHORT VERSION

Everything Terminal 3 built works. Registration, agent authorization, TEE dispatch, KV access control, egress allowlisting, outbound HTTP — all verified against the live testnet node. What does not work is the path a new developer is told to walk: the Quickstart snippet throws, and the only way past it is to switch off the attestation verification the product exists to provide.

What completed

STAGE	RESULT	DETAIL
Claim API key + DID	pass	DID resolved on testnet
Quickstart — authenticate	pass	after supplying an undocumented required field
Quickstart — credit balance	fail	broken on all four published surfaces
Walkthrough — write contract	pass	Rust crate, `contracts` WIT world
Walkthrough — build	pass	194 KB component in 34 s
Walkthrough — test	pass	7 / 7, after working around the documented command
Walkthrough — register	pass	`contract_id 511`
Walkthrough — invoke	pass	full TEE round trip to a live external API

The proof is an error message

The final invocation returns an HTTP 401. That is the success condition, and it is worth being precise about why.

```
$ node --env-file=.env src/invoke.ts
tenant/agent DID: did:t3n:6ec29eeb5cb122d05e006391d2c954b2390032ed
script name     : z:6ec29eeb5cb122d05e006391d2c954b2390032ed:flight
OK   agentAuthUpdate
FAIL search-offers: RpcError: contract error:
     Duffel offer-request failed: HTTP 401 - {"errors":[{
       "title":"Access token not found",
       "type":"authentication_error",
       "code":"access_token_not_found"}],
     "meta":{"request_id":"GMnF1mMwiWcBR68BZJ2I","status":401}}
```

The rejection comes from Duffel, not from Terminal 3 — it is refusing a placeholder token I supplied deliberately. To generate that refusal, the platform had to dispatch the contract into the enclave, instantiate the WASM component, resolve its host capabilities, read a secret out of an access-controlled KV map, and perform real outbound HTTPS through the egress allowlist. Every link in the T3 chain held. Only the third-party credential was fake.

Sixteen bugs

Ordered by severity. Every entry in the repository carries a repro, expected-versus-actual, and a workaround where one exists.

BUG-002
CRITICAL

The reference contract's README and its WIT document **opposite** PII security models.

README: `book-offer` posts "with full passenger PII … passed in by the agent." WIT: it "Carries NO passenger PII," using host-resolved `{{profile.*}}` placeholders. The crate's own passing test `book_offer_rejects_inline_pii_fields` settles it — the WIT is right, and the README teaches the one pattern the product exists to prevent.

BUG-011
CRITICAL

The published Quickstart code does not run, and the only available fix disables attestation.

`trustAnchor` is a required field; the snippet omits it, so `handshake()` throws. A real anchor needs `expected_peer_ids` and `rtmr3_allowlist` — values published nowhere. The only way through is `unsafe_trust_server: true`, which the SDK itself calls the only way to skip DKG attestation verification.

BUG-013
HIGH

Credit balance is unreachable from all four published surfaces.

SDK `getBalance()`, SDK `getUsage()`, CLI `token balance`, CLI `token usage` — all fail. The client seals `token.get-usage` params while the node expects a plain struct. Nobody can confirm they received the 20,000 credits the sandbox offer is built on.

BUG-012
HIGH

`getBalance()` throws inside the SDK's own decrypt path.

`InvalidCharacterError` at `atob` in `SessionEncryption.decrypt` — decrypting a response that was never sealed.

BUG-010
HIGH

Invocation requires an undocumented KV map whose ACL is keyed on the numeric contract id.

Three steps appear in no documentation: create the map, set `readers` to `{ only: [511] }`, seed the value. An omitted `readers` silently means deny-all. This is the real reason to keep every `contract_id` — and the page that issues them never says so.

BUG-005
HIGH

The sample's capability manifest omits a capability its own WIT imports.

The manifest lists four capabilities; the world also imports `http-with-placeholders`, so `book-offer` cannot dispatch as documented.

BUG-003
HIGH

Registering a new version silently re-routes previously pinned versions.

Documented as a caveat. A pinned version that moves is a defect — the failure is silent and the suggested mitigation is bookkeeping the platform should do itself.

BUG-014
HIGH

A fresh SDK install reports a critical Zip Slip advisory.

`decompress` via `weval → componentize-js → jco`. Regardless of exploitability, it fails supply-chain review at exactly the regulated enterprises this product targets.

BUG-006
MEDIUM

A pinned wasm target in `.cargo/config.toml` breaks the documented test command.

`cargo test --lib` builds tests for wasm then tries to execute them natively. Workaround: pass an explicit native `--target`; 7/7 then pass.

BUG-016
MEDIUM

`tenant.claim()` returns a bare `Internal error`.

The SDK defines an `already-admitted` result for exactly this case, but the node returns an opaque 500 instead, on the credit-claiming path.

BUG-015
MEDIUM

The shipped SDK is obfuscated with no source maps.

Every frame resolves to `index.esm.js:2:<column>`; one thrown error printed 1.2 MB of minified source. For a product asking enterprises to trust it with key custody, unauditable code is a posture problem as much as a DX one.

BUG-009
MEDIUM

The API key is unrecoverable and has no documented rotation path.

Shown once. A leaked key appears to mean abandoning the DID and its credits.

BUG-004
LOW

The docs state the wrong default environment — for both the SDK and the CLI.

Docs say production in three places; the CLI's own help and the SDK's `getEnvironment()` both say `testnet`. The documented environment list also omits `sandbox`, which both accept.

BUG-007

LOW

Three different versions inside one sample repository.

README says 0.3.0, `Cargo.toml` says 0.4.1, the WIT package says 0.4.0 — and registration demands you pick one.

BUG-001

LOW

Three names for the environment across three surfaces.

`sandbox`, `testnet`, and a CLI list containing neither consistently. Verified harmless — the two names share one URL — but nothing documents that.

BUG-008

LOW

"No Rust, WASM, or blockchain knowledge required" is contradicted by the next page.

Which immediately requires `rustup target add wasm32-wasip2`, WIT worlds, and `wit-bindgen`.

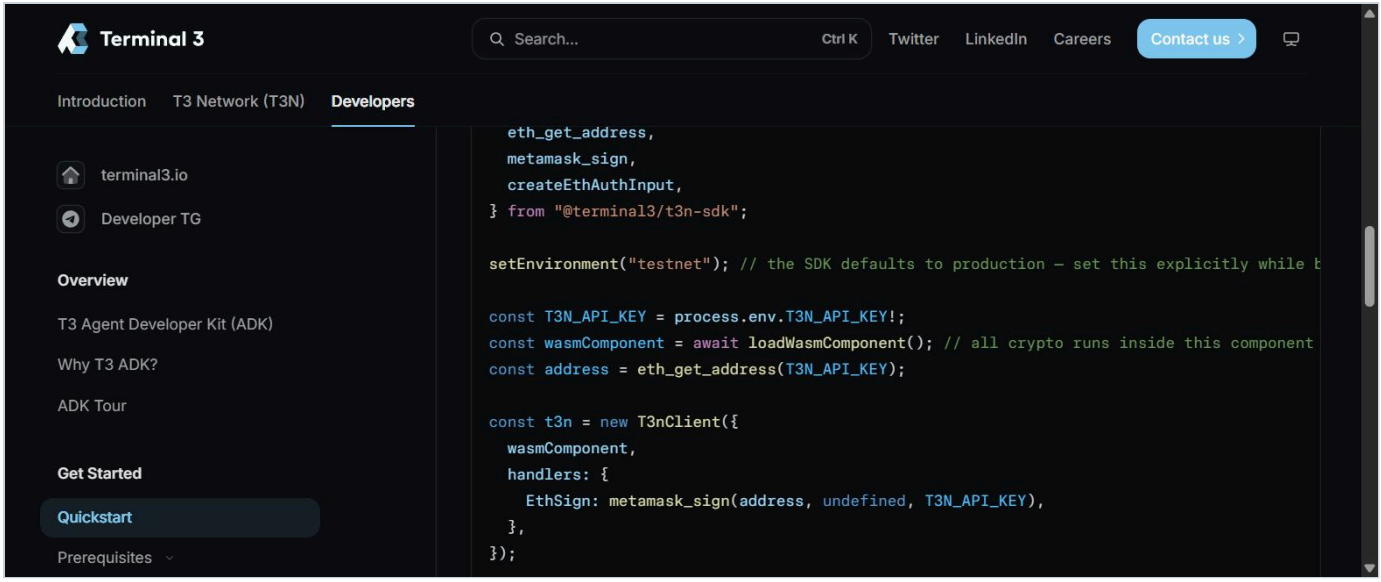
ON SEVERITY DISCIPLINE

Two of these were filed wrong and corrected downward after verification. BUG-001 was filed Critical on the assumption that `sandbox` and `testnet` were different environments; the SDK showed they resolve to one URL. BUG-004 was filed claiming the CLI dangerously defaults to production; running it showed the opposite — the default is safe and the docs are simply wrong.

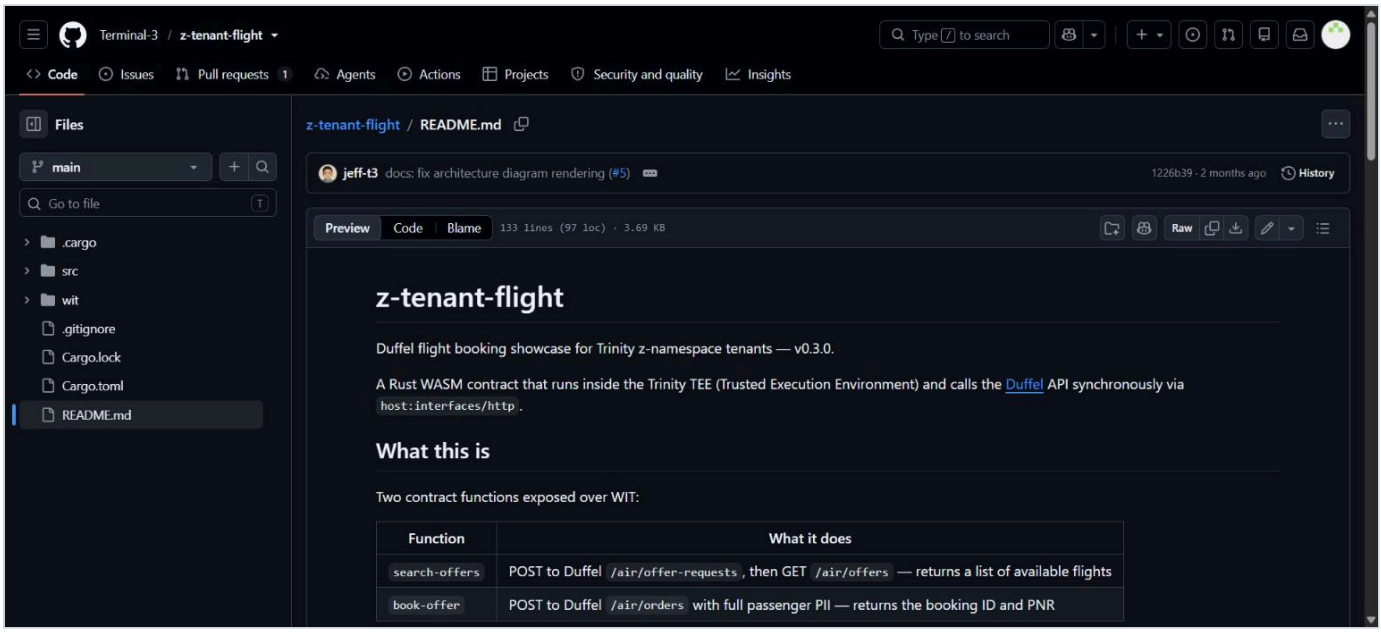
Both corrections are recorded in the repository with the original reasoning intact rather than quietly edited. A bug report is only worth reading if its severities can be trusted, and the cheapest way to establish that is to show the ones that moved.

Evidence

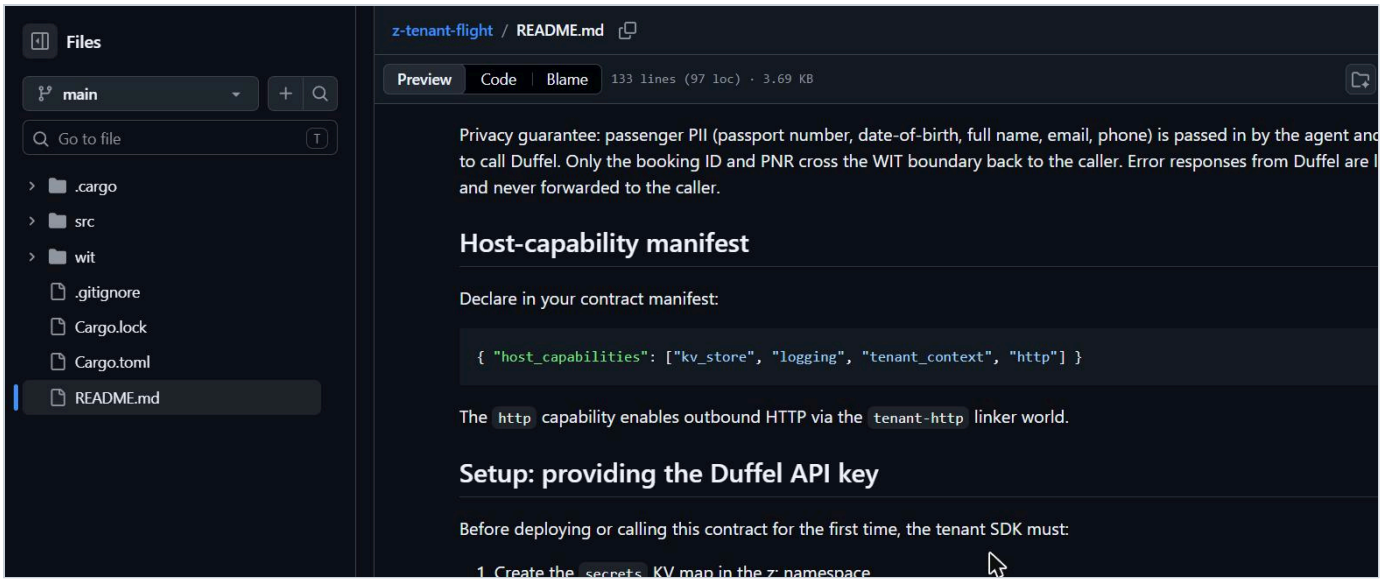
Captured live from Terminal 3's own published pages. These are the source documents, not a reconstruction of them.



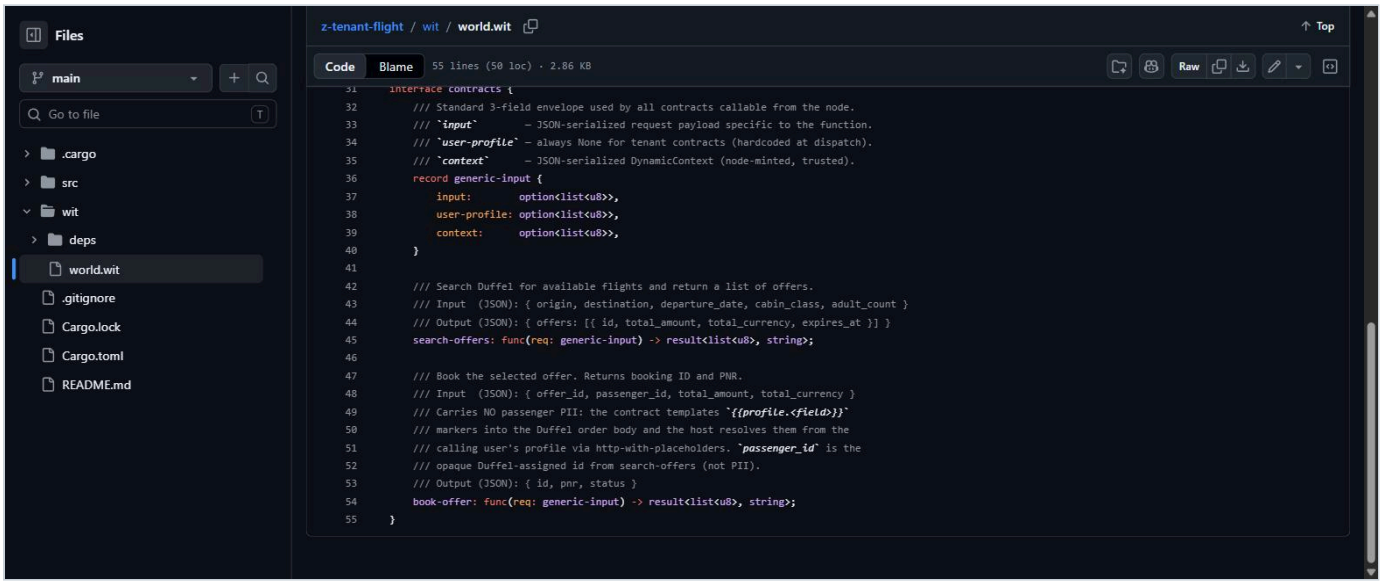
BUG-011. The published Quickstart constructor — `wasmComponent` and `handlers`, no `trustAnchor`. This is the code that throws. The inline comment claiming the SDK defaults to production is also false.



BUG-002 and BUG-007. The README describes `book-offer` as posting "with full passenger PII", under a version heading that disagrees with both `Cargo.toml` and the WIT package.



BUG-002 and BUG-005. The privacy guarantee claims PII "is passed in by the agent" — and the capability manifest immediately below it omits `http_with_placeholders`, the very mechanism that would make that unnecessary.



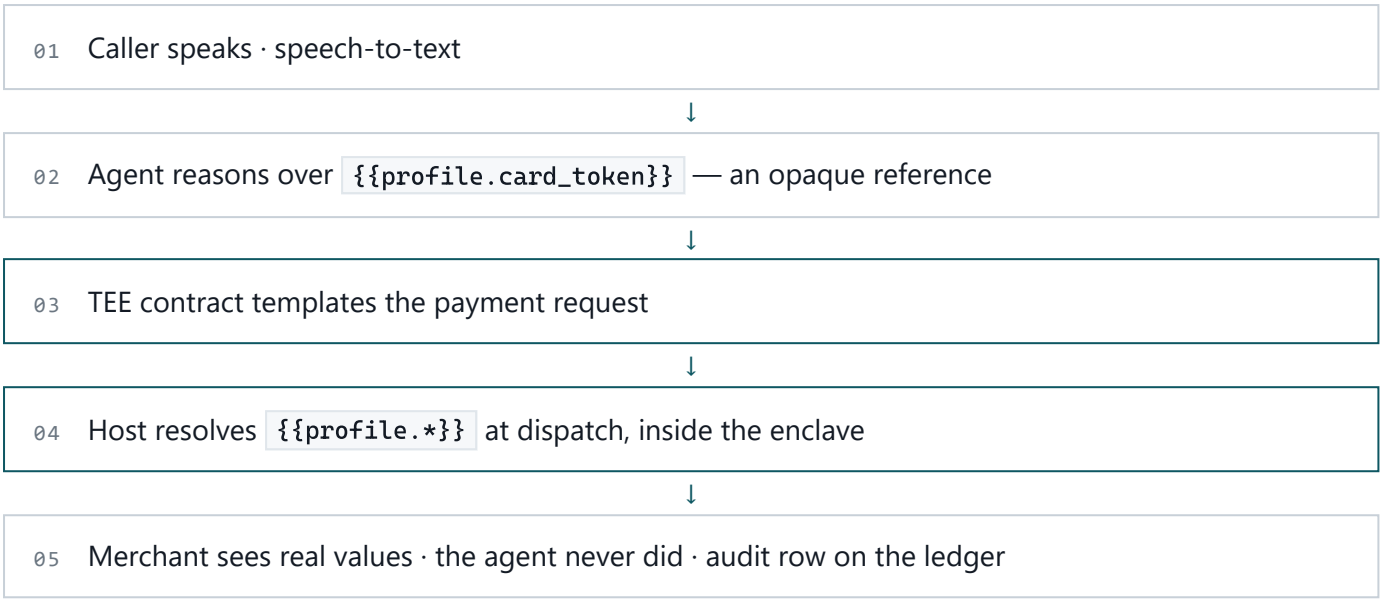
BUG-002, the other half. Line 49 of `world.wit`: "Carries NO passenger PII." Two documents in one repository, describing opposite security models for the same function.

Beyond the first contract

The reference contract protects passenger data in a flight booking. The same primitive solves a problem I keep hitting in my own work building voice agents of my own.

When a caller reads a card number aloud to a voice agent, that number lands in the speech-to-text transcript, the model's context window, and every log and trace downstream. Redacting afterwards does not help — the data was already in the prompt. It is the single biggest reason voice agents do not touch payments.

`http-with-placeholders` inverts the problem. The agent never receives the card at all:



A contract exporting `authorize-payment` would template the card token and date of birth into a Stripe intent, with the caller's `agent-auth` grant bounding both the amount and the egress host. The model's context holds only references, so the transcript is safe to log, and the ledger row becomes the compliance artifact.

This maps directly onto Terminal 3's own call-centre demo, and it is what I intend to build next with the remaining credits.

Does it fit a live phone call?

A conversational turn has roughly 500-800 ms before it degrades. I measured T3N rather than assuming — 5 rounds against the registered contract on testnet.

PHASE	MEDIAN	MIN	MAX
Session establishment (handshake + auth)	1,515 ms	–	–
Contract dispatch, no egress	151 ms	96 ms	393 ms
Dispatch + real outbound HTTPS	437 ms	418 ms	455 ms

A warm invocation fits inside a single turn — but the session must be pre-warmed. At ~1.5 s, handshake and authentication cannot happen mid-call; the agent needs an authenticated session open while the phone is still ringing. Dispatch overhead alone is only 151 ms, so the enclave is not the bottleneck — network egress is. That is a good result for Terminal 3: confidential computing is not what costs you the conversation.

The unsolved part is barge-in during an in-flight authorization: if the caller says *cancel that* 200 ms into a 437 ms payment, the payment completes anyway. The honest design is not to cancel mid-flight but to keep the window small and treat every authorization as reversible for a bounded period.

What would have made this faster

- **Publish testnet trust-anchor values.** One paragraph removes the worst bug here, and stops every new developer from disabling attestation as their first act.
- **Fix `token.get-usage`.** The sandbox's headline offer is 20,000 credits that currently cannot be observed.
- **Add the KV map and ACL step to the invoke page.** Three commands, currently zero words.
- **Ship source maps.** Isolating these bugs meant reading type declarations because the implementation is unreadable.

Hitansh Gopani · gopanihitansh5@gmail.com · @Hitansh54 · 8 August 2026

Full bug detail, verbatim terminal transcripts, and reproduction instructions: github.com/gopanihitansh5-collab/t3n-adk-submission